

Lab 4: Microservices Decomposition & Communication

This lab marks the transition from the monolithic Layered Architecture design (Lab 3) to the Microservices Architecture. The goal is to logically decompose the HRM application, define service boundaries, and establish how these independent services will communicate with each other (Synchronous via HTTP or Asynchronous via Redis Pub/Sub).

Objectives

1. Understand the principles of Microservices Decomposition by Business Capability.
2. Define clear Service Contracts (API Endpoints) for key services.
3. Design the system's High-Level Communication Strategy (Synchronous vs. Asynchronous).
4. Model the system using the C4 Model (Level 1: System Context).

Tool & Technology Setup

Tool	Purpose	Installation/Setup Guide
draw.io (Diagrams.net)	Creating C4 Models and high-level communication diagrams.	Access online via a web browser (built-in C4 Model shape library available).
Documentation Tool (Word/Docs)	Documenting Service Contracts (API Specifications).	Use standard word processing software.
No Coding Required	This lab is purely for architectural design and documentation.	N/A

Activity Practice 1: Decomposition by Business Capability

Goal: Identify clear, independent microservices based on the business functions of the HRM system.

Step-by-Step Instructions:

1. **Identify Core Business Capabilities:** Review the project report and group functionalities into independent business domains.

- *Action:* List the primary domains (e.g., Identity/HR Management, Leave Management, Payroll Management, Internal Communication).
2. **Define Microservices:** Map each capability to a dedicated microservice. Ensure services are loosely coupled (if one service goes down, the others can still partially function).

Business Capability	Proposed Microservice	Entities Owned
Identity & HR Management	Auth/User Service	User Profile (Accounts), Authorization (RBAC), Departments, OTP Codes.
Scheduling & Leave	Leave Service	Leave Requests, User Schedules (Shifts).
Attendance & Payroll	Salary Service	Attendance Logs, Monthly Payrolls, Bonuses/Penalties.
Communication & Notification	Notification Service	Announcements (General notices), Read Receipts, Online Status.

3. **Identify External Dependencies:** Determine which external systems the HRM needs to interact with.
- *Action:* The system sends SMTP Emails (NodeMailer) for OTPs, and uses a Real-time Socket.io Client provider. (In the future, a Geolocation API might be added for attendance tracking).

Activity Practice 2: Defining Service Contracts

Goal: Establish the public interfaces (API Endpoints) that services will expose to the API Gateway (Nginx) or to other calling services.

Step-by-Step Instructions & Documentation:

Focus on the interaction between the **Leave Service (Producer)** and the **Salary Service (Consumer)**. *Architectural Note: In a standard Microservices model, the salary_service is not allowed to query the leave_requests table in the Database directly; it must call it via an API.*

1. **Define the Leave Service API:** Document the RESTful Contracts.

Endpoint	HTTP Method	Description	Data Returned
/api/leaves	POST	Create a new leave request (Checks > 3 days, > 5 people rules).	LeaveRequest Object (with PENDING status)
/api/leaves	GET	Get a list of leave requests (Can filter by ?status=APPROVED).	Array of LeaveRequest Objects
/api/leaves/schedule/weekly	GET	Returns a 7-day Work Schedule matrix for personnel.	Schedule Object (Contains daily work/leave status)
/api/leaves/:id/status	PUT	Manager/Admin Approves or Rejects a leave request.	Success message and updated request data

2. **Define the Salary Service Interaction:** Determine the information the Salary Service needs from the Leave Service when finalizing the monthly payroll.

- *Action:* For the syncPayrollRealtime function to calculate salary, the Salary Service needs to know if the employee had valid leave (paid or unpaid) during that month.
- *Constraint:* The Salary Service must call GET /api/leaves?status=APPROVED&month=... from the Leave Service instead of writing an SQL SELECT * FROM leave_requests query.

Activity Practice 3: C4 Model (Level 1: System Context)

Goal: Create a high-level visual map showing the boundaries of the new system and its external interactions.

Step-by-Step Instructions (Using draw.io with C4 shapes):

1. **Draw the System:** Create a large rectangular block in the center, labeled [System] Human Resource Management System (HRM Platform).
2. **Place Users and External Systems:**

- Draw the "Employee (STAFF)" and "Manager / Director (ADMIN/MANAGER)" actors outside the system on the left.
- Draw the "SMTP Email Server" (External System) on the right.

3. Define Communication Flows: Draw directional arrows:

- Employee $\rightarrow$ HRM System: "Logs attendance, Views payroll, Submits leave requests".
- Manager $\rightarrow$ HRM System: "Approves requests, Views HR statistics, Sends announcements".
- HRM System $\rightarrow$ SMTP Email Server: "Sends OTP verification codes and Notification emails".

4. Analyze Low-Level Communication Strategy (Internal to the System): Differentiate between synchronous (HTTP) and asynchronous (Redis Pub/Sub & Socket) communication needs.

Interaction	Between Services / Components	Communication Type	Architectural Rationale
Login & Fetch Info	React Client (via Nginx) $\rightarrow$ Auth Service	Synchronous (HTTP REST)	Requires immediate response to retrieve the JWT Token and render the UI.
Check Leave Balance	Nginx $\rightarrow$ Leave Service	Synchronous (HTTP REST)	Needs remaining leave days data immediately upon opening the registration form.
New Request / Approval Alert	Leave Service $\rightarrow$ Redis $\rightarrow$ Noti Service (Socket.io)	Asynchronous (Event-Driven / Pub-Sub)	The request approval process (Saving to DB) is not blocked/slowed down by sending notifications. Notifications are pushed silently to the manager/employee screens (Real-time).
Send OTP Email	Auth Service $\rightarrow$	Asynchronous (Background Task)	Sending emails has high network latency; the user's registration flow

	NodeMailer (SMTP)		must not "hang" waiting for the email to finish sending.
--	----------------------	--	---